

Der Kochwettbewerb zwischen Debian und Arch

Debian und Arch: Zwei Linux-Distributionen und die größten Rivalen in der Welt der Pinguine. Seit Jahrhunderten messen sich die besten Koch-Pinguine der beiden Clans bei einem alljährlichen Kochwettbewerb.

Der diesjährige Wettbewerb steht bevor und es geht in die Auswahl der Pinguine, die antreten werden. Die Vorbereitungen laufen auf Hochtouren.

Du bist von den Mentor:innen des Debian-Clans beauftragt worden, ein Tool zu bauen, mit dem sie so gut wie möglich auswählen können, welche Pinguine sie in den Wettbewerb schicken.

(Kontext: Das Maskottchen von Linux heißt "Tux" und ist ein Pinguin.)

Rahmenbedingungen

Die Mentor:innen haben gezielt dich ausgewählt, weil sie wissen, wie gut du dich mittlerweile mit JavaScript auskennst; und genau deshalb wollen sie auch, dass das Tool **ausschließlich von dir umgesetzt** wird. Es gelten strenge Anforderungen an deine Arbeitsumgebung:

- Keine Unterlagen, Hilfestellungen oder bereits gelöste Übungen
- Dein Editor darf nur dieses Live Coding zeigen
- Dein Browser darf nur zum Darstellen dieses Live Codings verwendet werden (inkl. Endpunkt, von dem die Daten ge-fetched werden)
- Alle AI-Features im Editor und Browser sind deaktiviert

Vorgehensweise

- Die Live Coding Challenge besteht aus zwei Pflichtaufgaben und einer Bonusaufgabe.
- Die Bearbeitungszeit beträgt 90 Minuten.

Abgabe im Repo und im Webpace

Zwischenabgabe

Committe und pushe nach jedem Teil den aktuellen Stand in dein Repository.

Endabgabe

Lade dein Projekt in den Ordner /linux-kochbewerb deines Webspaces hoch und pushe den finalen Stand in einem Ordner mit demselben Namen ins Abgabe-Repository.

Setup

Entpacke die Zip-Datei, es entsteht ein Ordner /linux-kochbewerb/. Kopiere diesen Ordner in deinen WebProg2Abgabe/-Ordner.

Enthalten sind

Datei	Inhalt
index.html	HTML Template
css/styles.css	CSS Template
css/fonts.css	Definiert Schriftarten
fonts/	Schriftdateien
images/	Enthält die Bilder der Pinguine, deren Pfad du in den Objekten findest

Anforderungen

1. Teilnahmeberechtigte Pinguine auflisten

Um die besten Pinguine auswählen zu können, brauchen die Mentor:innen erst einen Überblick über die Kandidaten. Dafür hat das Verwaltungsteam des Büro 321 an der FH Salzburg folgenden

👉 Endpunkt:

<https://users.ct.fh-salzburg.ac.at/~fhs47779/cooking-competition/penguins.php>

bereitgestellt, der einen Überblick über alle Pinguine gibt, die auf Andreas Bilkes Schreibtisch zu finden sind. Alle dieser Pinguine sind teilnahmeberechtigt.

Als **Array aus Objekten** findest du unter dem Endpunkt alle Pinguine in folgendem Format:

```
{
  "name": "Name des Pinguin",
  "backstory": "Kurze Zusammenfassung des Werdegangs dieses Pinguins",
  "imageUrl": "./images/captain-tux.png"
}
```

`imageUrl` -> enthält einen relativen Pfad. Achte auch darauf, dass dieser Pfad am Server funktioniert.

Erstelle folgende Dateien:

1. **script.js**

Importiert **getPenguins()** und instanziiert das TeamSelectionTool. Wird in die index.html eingebunden.

2. **tool.js**

Enthält die Klasse TeamSelectionTool mit folgender Signatur:

```
class TeamSelectionTool {  
  constructor(penguins, parentElement) {}  
  displayPenguins() {}  
}
```

penguins ist ein Array mit den Daten vom Endpunkt.

parentElement ist das Element mit der id="tool-wrapper".

Rufe im Konstruktor **this.displayPenguins()** auf.

Diese Funktion zeigt alle Pinguine in <div id="penguins-wrapper"> an.

Für jeden Pinguin wird ein <div class="penguin-card"> erstellt mit:

- — imageUrl als src
- <h2> — Name
- <p> — Backstory
- <button class="select-button"> — Text „Auswählen“

Füge jedes <div> mit .appendChild() oder .append() der Liste id="list-wrapper" an, die im HTML Template bereits vorhanden ist.

3. **getPenguins.js**

Exportiert die Funktion:

```
async function getPenguins() {}
```

Fetcht die Daten vom Endpunkt mit *fetch()* und gibt das Array als JSON zurück.

Korrekte Verwendung von *async/await* beachten.

Styling kann verändert werden, aber die Klassennamen müssen erhalten bleiben.

2. Darstellen der ausgewählten Pinguine

Wenn ein:e Mentor:in auf "Auswählen" klickt, soll der jeweilige Pinguin im `<div id="selected-penguins-wrapper">` angezeigt werden, das bereits im HTML-Template vorhanden ist.

1. Implementiere die Funktion

```
selectPenguin(penguin)
```

Sie wird aufgerufen, wenn man auf "Auswählen" klickt. Speichere die ausgewählten Pinguine in einem separaten Array, das **team** heißt.

Prüfe zuerst: Ist der Pinguin bereits im Team → Wenn ja, return (nichts tun)
Andernfalls: Füge penguin mit **this.team.push(penguin)** zum Team hinzu

2. Implementiere die Funktion

```
displaySelectedPenguins()
```

Gehe durch das Array **this.team** und erstelle *für jeden* Pinguin in diesem Array ein `<div>` mit einem `<img>` und einer `<h2>` mit dem Namen.

Gib dem `<div>` folgende Klassen:
`element.classList.add("penguin-card", "team-member")`

Füge jedes `<div>` mit `.appendChild()` oder `.append()` zu `<div id="selected-penguins-wrapper">` hinzu.

3. BONUS: Limitierte Plätze

Die Mentor:innen dürfen denselben Pinguin nicht doppelt setzen. Es gibt drei Runden, also maximal *drei* Pinguine dürfen ausgewählt werden.

- Setze im Konstruktor **this.MAX_COOKS_ALLOWED = 3**. Verwende diese Variable überall dort, wo du prüfst, ob das Team noch Platz hat.
- Wurde ein Pinguin bereits ausgewählt, wird sein Button deaktiviert (*button.disabled = true*).

Optional: Der Text ändert sich zu "Bereits ausgewählt" und der Pinguin wird grün eingerahmt — verwende dafür die Klasse `.penguin-selected`.

- Wenn alle vier Plätze belegt sind, werden alle verbleibenden Buttons deaktiviert
 Tipp: `.querySelectorAll("select-button")` an die wrapper-list gehängt returnt alle Buttons die du dafür brauchst als Array.
- Füge oberhalb von `<div id="selected-penguins-wrapper">` ein `<p class="team-status">` Element ein. Solange das Team noch nicht voll ist, zeigt es an, wie viele Plätze noch frei sind. Ist es voll, wird stattdessen angezeigt: "Das Team steht fest. Viel Erfolg beim Bewerb!"

Füge in diesem Fall zusätzlich die Klasse **team-full** hinzu. Beide Klassen sind bereits im CSS definiert.

Tipps Tipps Tipps!

1. `array.push()` fügt ein neues Element an array an.
2. Um einem Element eine farbliche Umrandung zu geben, verwende zum Beispiel `element.style.border = "2px solid #000";`
Du kannst width style und color setzen, auch separat,
z.B.mit `element.style.borderColor = "black"`
3. `console.log()` zum Debuggen verwenden!!

Dateiliste

sowohl am Webspace, als auch im Repository

Datei	Hinweis
index.html	-
/css	Mindestens styles.css und fonts.css
/scripts/script.js	Instanziierung vom TeamSelectionTool
/scripts/getPenguins.js	Fetcht Pinguine vom definierten Endpunkt
/scripts/tool.js	Implementiert die TeamSelectionTool Klasse

Viel Erfolg! Wer wird den diesjährigen Kochbewerb gewinnen?